



**UNIVERSITÉ
DE GENÈVE**



**UNIVERSITÉ
DE GENÈVE**

FACULTÉ DES SCIENCES

UNIVERSITÉ DE GENÈVE

FACULTÉ DES SCIENCES

Département d'Informatique

Travail de Bachelor en Sciences Informatiques

Année académique 2025-2026

Programmation dynamique pour formules SAT

Auteur :

Elie BUSSOD

Superviseurs :

Prof. Arnaud CASTEIGTS

Prof. Pierre LEONE

Date de rendu : Juin 2026

Résumé

Bla bla abstract

Table des matières

Résumé	1
1 Introduction	5
1.1 Contexte et motivation	5
1.2 Approches algorithmiques	5
1.3 Approche structurelle et décomposition arborescente	6
1.4 Objectifs de ce travail	6
1.5 Plan du rapport	6
2 État de l’art	8
2.1 SAT, #SAT et MaxSAT	8
2.1.1 Le problème SAT	8
2.1.2 #SAT : comptage de modèles	8
2.1.3 MaxSAT	9
2.2 Algorithmes de résolution	9
2.2.1 DPLL et propagation unitaire	9
2.2.2 CDCL : apprentissage de clauses guidé par les conflits	9
2.2.3 Solveurs #SAT	9
2.2.4 Solveurs MaxSAT	9
2.2.5 SMT et compilation de connaissances	9
2.3 Décompositions arborescentes et paramètres de largeur	10
2.3.1 Treewidth	10
2.3.2 Algorithmes DP paramétrés par la treewidth	10
2.3.3 Mim-width	10
2.4 PS-width et l’algorithme de Sæther-Telle-Vatshelle	11
2.4.1 Branch decomposition et PS-sets	11
2.4.2 L’algorithme DP	11
2.4.3 Heuristique GreedyOrder et instances de référence	11
2.5 Backdoors pour SAT	12
2.5.1 Définition	12
2.5.2 Détection et heuristiques	12
2.6 Circuits de multiplication et transformation de Tseytin	12
3 Cadre théorique	13
4 Implémentation	14
5 Résultats expérimentaux	15
6 Discussion	16
7 Conclusion	17

A Annexes	18
Bibliographie	19
Déclaration d'utilisation de l'IA	22

Table des figures

1

Introduction

1.1 Contexte et motivation

La satisfaisabilité propositionnelle (SAT) occupe une place centrale en informatique théorique et appliquée depuis les travaux de Cook [1] et Karp [2], qui ont établi sa NP-complétude et en ont fait le problème de référence de la théorie de la complexité. Malgré cette intractabilité théorique dans le cas général, les solveurs modernes parviennent à traiter des instances industrielles comportant des millions de variables et de clauses, grâce notamment à l'algorithme d'apprentissage de clauses guidé par les conflits (CDCL) [6, 5], à la propagation unitaire (BCP) et à des heuristiques de branchement sophistiquées. Cette efficacité pratique a conduit à une adoption massive de SAT dans des domaines aussi variés que la vérification formelle de circuits électroniques, la planification automatique, la synthèse de programmes et la cryptanalyse.

Ce succès a naturellement suscité un intérêt croissant pour des variantes plus expressives : le *comptage de modèles* (#SAT) et la *maximisation de clauses satisfaites* (MaxSAT). Le problème #SAT consiste à déterminer le nombre exact d'affectations des variables qui satisfont une formule CNF donnée. Il généralise la décision de satisfaisabilité en quantifiant la densité de l'espace de solutions, et intervient naturellement dans le raisonnement probabiliste, l'évaluation de réseaux bayésiens et la vérification de probabilités [10]. Le problème MaxSAT, quant à lui, cherche une affectation qui satisfait le plus grand nombre possible de clauses, même lorsque la formule est globalement insatisfaisable. Il modélise des problèmes d'optimisation combinatoire à contraintes souples (ordonnancement, configuration, bioinformatique) et constitue un cadre unifiant pour de nombreux problèmes NP-difficiles.

Ces deux variantes sont rigoureusement plus difficiles que SAT : #SAT est #P-complet [3] et MaxSAT est NP-difficile. Cette complexité accrue justifie le développement d'approches algorithmiques spécialisées.

1.2 Approche structurelle et décomposition arborescente

Plusieurs familles d'algorithmes ont émergé pour résoudre #SAT et MaxSAT : solveurs généraux fondés sur DPLL et CDCL, outils SMT comme Z3 [9], solveurs de comptage avec cache de composantes comme **sharpSAT** [7], ou encore la *compilation de connaissances* [10] qui traduit la formule hors-ligne en une représentation canonique (d-DNNF, SDD) permettant ensuite des requêtes #SAT en temps polynomial. Ce travail se concentre principalement sur une perspective différente, issue de la complexité paramétrée : l'exploitation de la *structure* de la formule.

L'idée fondatrice est que la difficulté d'une instance ne dépend pas uniquement de sa taille $n + m$ (variables et clauses), mais de paramètres structurels liés à la façon dont variables et clauses interagissent. Si ces paramètres sont petits, des algorithmes par *programmation dynamique sur décomposition arborescente* résolvent #SAT et MaxSAT en temps polynomial en $n + m$, mais exponentiel uniquement en le paramètre structurel.

Parmi ces paramètres, la *treewidth* du graphe d'incidence de la formule est le plus étudié [14, 15]. Des résultats plus récents, fondés sur la *mim-width* [16] et la *ps-width* [18], étendent la tractabilité à des classes de formules strictement plus larges. L'algorithme de Sæther, Telle et Vatshelle [18] résout simultanément #SAT et MaxSAT en temps $O(k^3 \cdot m \cdot (m + n))$, où k est la *ps-width* de la décomposition, et constitue l'algorithme central de ce travail.

1.3 Objectifs de ce travail

Ce mémoire de bachelor poursuit deux objectifs complémentaires.

Le premier est l'**implémentation complète** de l'algorithme de Sæther, Telle et Vatshelle [18] en langage C. Cette implémentation comprend le parseur de formules DIMACS CNF, quatre stratégies de construction de l'arbre de décomposition (dont l'heuristique GreedyOrder décrite dans le papier), les trois procédures de l'algorithme (calcul bottom-up des PS-sets, propagation top-down, programmation dynamique), ainsi que les structures de données sous-jacentes (bitsets 64-bit, trie binaire, tables DP).

Le second objectif est l'**évaluation empirique** sur une variété de formules : instances synthétiques conçues pour contrôler la *ps-width*, formules aléatoires k -SAT, et formules CNF issues de circuits de multiplication encodés par la transformation de Tseytin. Ces expérimentations visent à mesurer l'impact de la *ps-width* sur les performances, à comparer l'algorithme DP avec d'autres approches (solveurs CDCL, Z3, compilation de connaissances), et à identifier les conditions dans lesquelles la décomposition structurelle apporte un avantage décisif. Nous explorons également la notion de *backdoor fort*, un petit ensemble de variables dont la fixation ramène la formule résiduelle à une *ps-width* gérable, pour étendre la portée pratique de l'algorithme DP.

2

État de l'art

Ce chapitre positionne notre travail dans la littérature. Nous présentons successivement les problèmes SAT, #SAT et MaxSAT, les principales familles d'algorithmes de résolution, les paramètres de décomposition structurelle qui gouvernent leur complexité, l'algorithme de Sæther, Telle et Vatshelle qui constitue le socle de notre implémentation, et enfin la notion de backdoor qui en étend la portée.

2.1 SAT, #SAT et MaxSAT

2.1.1 Le problème SAT

Le problème de satisfaisabilité propositionnelle (SAT) demande, étant donnée une formule F en forme normale conjonctive (CNF), s'il existe une affectation des variables qui la satisfait. Rappelons qu'une formule CNF est une conjonction de clauses, chaque clause étant une disjonction de littéraux (variable ou sa négation). Une affectation satisfaisante est appelée *modèle* de F .

SAT fut le premier problème formellement démontré NP-complet par Cook [1], puis Karp en étendit l'applicabilité en montrant que de nombreux problèmes combinatoires classiques s'y réduisent en temps polynomial [2]. Malgré cette intractabilité théorique, les solveurs industriels modernes traitent couramment des instances de plusieurs millions de variables et clauses, grâce à l'algorithme CDCL [4, 5] décrit à la section 2.2.1.

2.1.2 #SAT : comptage de modèles

Le problème #SAT (*model counting*) demande non plus si un modèle existe, mais *combien* il en existe. Ce problème est #P-complet [3], une classe de complexité qui capture les problèmes de comptage associés aux décisions NP. Valiant a notamment montré que même sur des fragments syntaxiques pour lesquels la décision est facile (2-CNF, Horn), le comptage reste #P-complet.

#SAT intervient naturellement dans l'inférence probabiliste sur des réseaux bayésiens [10], le model checking quantitatif, et l'analyse de fiabilité de systèmes. Sa résolution exacte exige d'explorer l'intégralité de l'espace de solutions, ce qui le distingue fondamentalement du problème de décision.

2.1.3 MaxSAT

Le problème MaxSAT cherche une affectation qui maximise le nombre de clauses satisfaites, même si la formule est globalement insatisfaisable. Il est NP-difficile et modélise des problèmes d'optimisation à contraintes souples — ordonnancement, configuration, bioinformatique. Des variantes importantes incluent le *Partial MaxSAT* (clauses dures obligatoires + clauses souples à maximiser) et le *Weighted MaxSAT* (poids par clause).

2.2 Algorithmes de résolution

2.2.1 DPLL et propagation unitaire

L'algorithme DPLL (Davis-Putnam-Logemann-Loveland) constitue le fondement historique de la résolution SAT : il parcourt en profondeur l'espace des affectations partielles en s'appuyant sur deux règles d'inférence principales. La *propagation unitaire* (BCP, pour *Boolean Constraint Propagation*) force la valeur d'un littéral dès qu'il est le dernier non falsifié d'une clause. La *règle du littéral pur* affecte d'emblée les variables n'apparaissant que sous une polarité. BCP est *confluent* : quel que soit l'ordre de traitement des clauses unitaires, le point fixe final est identique.

2.2.2 CDCL : apprentissage de clauses guidé par les conflits

L'architecture CDCL, popularisée par zChaff [4] puis MiniSAT [5], étend DPLL par l'apprentissage de clauses. Lors d'un conflit, le solveur analyse le graphe des implications causales pour extraire une *clause apprise* résumant la cause profonde de l'échec. Cette clause est ajoutée à la base et permet un *backjumping* non chronologique, éliminant des pans entiers de l'espace de recherche. La heuristique VSIDS (*Variable State Independent Decaying Sum*) [4] maintient un score d'activité par variable et oriente le branchement vers les variables impliquées dans les conflits récents.

2.2.3 Solveurs #SAT

Les solveurs #SAT étendent DPLL par le *cache de composantes* : lorsque l'affectation partielle déconnecte le graphe d'incidence en composantes disjointes, leurs comptes sont calculés indépendamment et multipliés. Le solveur **sharpSAT** [7] exploite cette propriété avec un encodage compact des composantes pour maximiser les réutilisations de cache, atteignant des performances sans commune mesure avec une exploration exhaustive naïve. Des solveurs plus récents comme D4 [8] combinent ce cache avec la compilation vers des circuits d-DNNF, rendant les requêtes ultérieures polynomiales.

2.2.4 Solveurs MaxSAT

Les solveurs MaxSAT modernes (**MaxHS**, **RC2**, **Open-WBO**) abandonnent le *branch-and-bound* traditionnel au profit d'approches guidées par les noyaux insatisfaisables (*core-guided*). **MaxHS** orchestre un dialogue entre un solveur SAT (extraction de noyaux) et un solveur ILP (calcul du *hitting set* minimal) pour converger vers la solution optimale avec des garanties de correction.

2.2.5 SMT et compilation de connaissances

Les solveurs SMT (satisfaisabilité modulo théories), dont Z3 [9], étendent le cadre SAT en intégrant des théories arithmétiques (arithmétique linéaire entière/réelle, fonctions non interprétées, vecteurs de bits). L'architecture DPLL(T) délègue la vérification de cohérence sémantique à

des *T-solveurs* spécialisés. Sur des instances purement propositionnelles, les solveurs SAT dédiés restent nettement plus rapides du fait de la surcharge des structures SMT.

La *compilation de connaissances* [10] adopte une perspective différente : la formule est transformée hors-ligne vers une représentation canonique (d-DNNF, SDD [12]) sur laquelle les requêtes #SAT s'exécutent en temps linéaire. Cette approche est efficace quand la base de connaissances est statique et interrogée fréquemment. Darwiche et Marquis ont formalisé une hiérarchie de ces langages en termes de succintitude et de requêtes supportées en temps polynomial [10].

2.3 Décompositions arborescentes et paramètres de largeur

2.3.1 Treewidth

Pour des formules issues du monde réel (vérification de circuits, planification), la difficulté pratique dépend moins de la taille brute que de la structure de connectivité. La *treewidth* (largeur d'arbre), introduite par Robertson et Seymour dans leur théorie des mineurs de graphes, mesure à quel point un graphe ressemble à un arbre.

Formellement, une *décomposition arborescente* d'un graphe $G = (V, E)$ est un couple (X, T) où T est un arbre dont chaque nœud i est associé à un *sac* $X_i \subseteq V$, vérifiant : (i) tout sommet de V appartient à au moins un sac ; (ii) toute arête $(u, v) \in E$ est couverte par un sac contenant les deux extrémités ; (iii) pour tout sommet v , les nœuds de T dont les sacs contiennent v forment un sous-arbre connexe. La *largeur* de la décomposition est $\max_i |X_i| - 1$, et la *treewidth* $tw(G)$ est le minimum sur toutes les décompositions possibles. Bien que calculer la *treewidth* optimale soit NP-difficile en général, le problème est FPT pour k fixé.

2.3.2 Algorithmes DP paramétrés par la treewidth

La *treewidth* du graphe d'incidence $I(F)$ d'une formule CNF (graphe biparti dont les sommets sont les variables et les clauses, reliés si la variable apparaît dans la clause) fournit un paramètre naturel pour des algorithmes DP. Samer et Szeider [14] ont proposé le premier algorithme DP correct pour #SAT paramétré par la *treewidth* incidente, de complexité $O^*(4^k)$ où k est la *treewidth*. Slivovsky et Szeider [15] ont ensuite amélioré ce résultat à $O^*(2^k)$ en reformulant l'étape de jonction des tables comme un produit algébrique calculé par transformée de Zeta et inversion de Möbius, atteignant la borne inférieure imposée par la conjecture SETH.

2.3.3 Mim-width

La *treewidth* ne suffit pas pour les graphes denses. La *mim-width* (*maximum induced matching width*), introduite par Vatschelle dans sa thèse de doctorat [16], mesure pour chaque coupe d'une décomposition par branchement la taille maximale d'un couplage induit dans le sous-graphe biparti défini par cette coupe. La *mim-width* est plus générale que la *treewidth* et la *clique-width* : des graphes d'intervalles ou de permutation, à *treewidth* non bornée, ont une *mim-width* bornée.

Cependant, Bergougnoux, Bonnet et Duron ont démontré en 2025 que calculer ou approximer la *mim-width* est un problème paraNP-complet [17]. En pratique, on recourt donc à des heuristiques pour construire des décompositions de bonne qualité.

2.4 PS-width et l'algorithme de Sæther-Telle-Vatshelle

2.4.1 Branch decomposition et PS-sets

Sæther, Telle et Vatshelle [18] introduisent un paramètre défini directement sur les formules CNF. Soit F une formule CNF à n variables et m clauses. Une *branch decomposition* (T, δ) est un arbre binaire T muni d'une bijection δ entre ses feuilles et les éléments de $\text{var}(F) \cup \text{cla}(F)$. Chaque nœud interne v partage F en deux sous-formules F_v (éléments dans son sous-arbre) et $F_{\bar{v}}$ (éléments à l'extérieur).

Pour une affectation τ des variables de F_v , on note $\text{sat}'(F_v, \tau)$ l'ensemble des clauses de $F_{\bar{v}}$ satisfaites par τ (clauses dont au moins un littéral, dont la variable est dans F_v , est rendu vrai par τ). L'ensemble de tous les sous-ensembles ainsi atteignables est :

$$PS(F_v) = \{ \text{sat}'(F_v, \tau) : \tau \text{ affectation de } \text{var}(F_v) \}.$$

La *ps-width* de la décomposition est le maximum de $|PS(F_v)|$ et $|PS(F_{\bar{v}})|$ sur tous les nœuds internes v , et la *ps-width* de F est le minimum sur toutes les décompositions.

Relation avec la mim-width. Sæther et al. établissent le corollaire clé : si le graphe d'incidence $I(F)$ a une *mim-width* w et si les clauses sont de taille au plus t , alors $\text{psw}(F) \leq \min(m^w + 1, 2^{tw})$. Une *mim-width* bornée garantit donc une *ps-width* polynomiale, et la *ps-width* domine la *mim-width* dans la hiérarchie des paramètres de décomposition.

2.4.2 L'algorithme DP

L'algorithme de [18] calcule #SAT et MaxSAT par programmation dynamique en trois procédures séquentielles sur la décomposition arborescente.

Procédure 1 (bottom-up). Pour chaque nœud v en ordre post-ordre, on calcule $PS'(F_v)$, une représentation comprimée de $PS(F_v)$, en combinant les PS-sets des deux enfants par une union filtrée : si c_1, c_2 sont les enfants de v , alors

$$PS'(F_v) = \{ (C_1 \cup C_2) \cap \text{cla}(F_{\bar{v}}) : C_1 \in PS'(F_{c_1}), C_2 \in PS'(F_{c_2}) \}.$$

Procédure 2 (top-down). On calcule symétriquement $PS'(F_{\bar{v}})$ depuis la racine vers les feuilles, en combinant la contribution du frère de v avec celle du parent.

Procédure 3 (DP). Pour chaque nœud v , on maintient une table Tab_v indexée par des paires $(C, C') \in PS'(F_v) \times PS'(F_{\bar{v}})$. $\text{Tab}_v(C, C')$ stocke le nombre maximal de clauses de $\delta(v)$ satisfaisables (MaxSAT) et le nombre d'affectations satisfaisantes (#SAT) cohérentes avec la projection (C, C') . Les tables sont mises à jour par les combinaisons de triplets (C_{c_1}, C_{c_2}, C'_v) en temps $O(k^3)$ par nœud.

La complexité totale est $O(k^3 \cdot m \cdot (m + n))$ où k est la *ps-width* [18]. Pour un arbre linéaire (chaque nœud interne a au moins un enfant feuille), l'un des PS-sets est de taille ≤ 2 , ce qui réduit la complexité à $O(k^2 \cdot m \cdot (m + n))$.

2.4.3 Heuristique GreedyOrder et instances de référence

Puisque minimiser la *ps-width* est intractable en général, les auteurs proposent l'heuristique *GreedyOrder* : à chaque étape, le sommet du graphe d'incidence $I(F)$ ayant le plus grand degré de connexion aux sommets déjà sélectionnés est choisi en premier (avec tie-break sur le degré total). Cette heuristique glouonne produit des décompositions linéaires de bonne qualité en pratique.

Pour valider l'algorithme, trois familles d'instances synthétiques sont utilisées : les formules *Type 1* (graphes bipartis d'intervalles), *Type 2* (mêmes structures avec clauses de petite taille) et

Type 3 (fonctions XOR cycliques formant des graphes d'arcs circulaires). Sur ces instances, le DP paramétré par la ps-width surpasse nettement les solveurs CDCL [18].

2.5 Backdoors pour SAT

2.5.1 Définition

La notion de backdoor, introduite par Williams, Gomes et Selman [19], identifie des sous-ensembles de variables dont la fixation ramène le problème dans une classe tractable. Formellement, un sous-ensemble $B \subseteq \text{var}(F)$ est un *backdoor fort* vers une classe $\mathcal{C}$ si, pour toute affectation $\tau : B \rightarrow \{0, 1\}$, la formule résiduelle $F[\tau]$ appartient à $\mathcal{C}$. Un *backdoor faible* exige seulement l'existence d'une affectation satisfaisante de B telle que $F[\tau] \in \mathcal{C}$.

Dans notre contexte, la classe cible est celle des formules à ps-width bornée : si pour tout τ , la ps-width de $F[\tau]$ est au plus k' , alors l'algorithme DP peut être appliqué sur chaque sous-problème. Le nombre total d'affectations est $\#SAT(F) = \sum_{\tau} \#SAT(F[\tau])$, et $MaxSAT(F) = \max_{\tau} [sat(\tau) + MaxSAT(F'[\tau])]$, où $sat(\tau)$ est le nombre de clauses satisfaites directement par τ .

2.5.2 Détection et heuristiques

Trouver le plus petit backdoor est NP-difficile. Nishimura, Ragde et Szeider [20] ont établi que la détection d'un backdoor de taille $\leq k$ vers Horn et 2-CNF est FPT. Gaspers et Szeider [21] ont étendu ce résultat aux backdoors vers la treewidth bornée. Ganian, Ramanujan et Szeider [22] ont introduit la notion de *backdoor treewidth*, exploitant la structure interne du backdoor plutôt que sa seule taille.

En pratique, la heuristique VSIDS fournit une sélection efficace : le score $score(v) = |C^+(v)| \times |C^-(v)|$ identifie les variables les plus « charnières », celles qui apparaissent équilibrément dans les deux polarités. Dilkina, Gomes et Sabharwal [23] ont montré que les backdoors exploitant l'activité CDCL (*learning-sensitive backdoors*) peuvent être exponentiellement plus petits que les backdoors statiques. Semenov et al. [24] ont appliqué ces idées à la cryptanalyse SAT, utilisant les bits de retenue des circuits arithmétiques comme candidats naturels au backdoor.

2.6 Circuits de multiplication et transformation de Tseytin

La transformation de Tseytin [25] convertit un circuit logique en une formule CNF équisatisfaisable en temps linéaire, en introduisant une variable auxiliaire pour la sortie de chaque porte. Cette transformation préserve la structure topologique du circuit dans le graphe d'incidence de la formule.

Pour un multiplicateur $k \times k$ bits (array multiplier), la formule Tseytin résultante comporte $O(k^2)$ variables et clauses. La structure en grille de l'additionneur génère un graphe d'incidence de mim-width $\Theta(k)$ [16], ce qui entraîne une ps-width exponentiellement grande en k . Pour $k \geq 4$, cette explosion rend l'algorithme DP pur intractable, ce qui motive l'approche par backdoor : les bits de retenue (carry bits), responsables des dépendances longue portée, constituent des candidats naturels au backdoor. Leur fixation par propagation unitaire propage en cascade les valeurs des portes adjacentes, effondrant la formule résiduelle en un sous-problème de ps-width nulle ou bornée.

3

Cadre théorique

4

Implémentation

5

Résultats expérimentaux

6

Discussion

7

Conclusion

A

Annexes

Bibliographie

Bibliographie

- [1] S. A. Cook. The complexity of theorem-proving procedures. In *Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pp. 151–158, 1971.
- [2] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.
- [3] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing*, 8(3) :410–421, 1979.
- [4] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff : Engineering an efficient SAT solver. In *Proc. 38th Design Automation Conference (DAC)*, pp. 530–535, 2001.
- [5] N. Eén and N. Sörensson. An extensible SAT-solver. In *Proc. 6th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 2919, pp. 502–518. Springer, 2003.
- [6] J. P. Marques-Silva and K. A. Sakallah. GRASP : A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5) :506–521, 1999.
- [7] M. Thurley. sharpSAT – Counting models with advanced component caching and implicit BCP. In *Proc. 9th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 4121, pp. 424–429. Springer, 2006.
- [8] J.-M. Lagniez and P. Marquis. An improved decision-DNNF compiler. In *Proc. 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 667–673, 2017.
- [9] L. de Moura and N. Bjørner. Z3 : An efficient SMT solver. In *Proc. 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340. Springer, 2008.
- [10] A. Darwiche and P. Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research (JAIR)*, 17 :229–264, 2002.
- [11] A. Darwiche. New advances in compiling CNF into decomposable negation normal form. In *Proc. 16th European Conference on Artificial Intelligence (ECAI)*, pp. 328–332, 2004.
- [12] A. Darwiche. SDD : A new canonical representation of propositional knowledge bases. In *Proc. 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 819–826, 2011.
- [13] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8) :677–691, 1986.
- [14] M. Samer and S. Szeider. Algorithms for propositional model counting. *Journal of Discrete Algorithms*, 8(1) :50–64, 2010.
- [15] F. Slivovsky and S. Szeider. A faster algorithm for propositional model counting parameterized by incidence treewidth. In *Proc. 23rd International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 12178, pp. 267–276. Springer, 2020.
- [16] M. Vatshelle. *New Width Parameters of Graphs*. PhD thesis, University of Bergen, 2012.
- [17] B. Bergougnoux, É. Bonnet, and J. Duron. Mim-width is paraNP-complete. In *Proc. 52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, LIPIcs, 2025. DOI : 10.4230/LIPIcs.ICALP.2025.25.

- [18] S. H. Sæther, J. A. Telle, and M. Vatshelle. Solving #SAT and MaxSAT by dynamic programming. *Journal of Artificial Intelligence Research (JAIR)*, 54 :59–82, 2015.
- [19] R. Williams, C. P. Gomes, and B. Selman. Backdoors to typical case complexity. In *Proc. 18th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 1173–1178, 2003.
- [20] N. Nishimura, P. Ragde, and S. Szeider. Detecting backdoor sets with respect to Horn and binary clauses. In *Proc. 7th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 96–103, 2004.
- [21] S. Gaspers and S. Szeider. Strong backdoors to bounded treewidth SAT. In *Proc. 54th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 489–498, 2013.
- [22] R. Ganian, M. S. Ramanujan, and S. Szeider. Backdoor treewidth for SAT. In *Proc. 20th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 10491, pp. 20–37. Springer, 2017.
- [23] B. Dilkina, C. P. Gomes, and A. Sabharwal. Backdoors in the context of learning. In *Proc. 12th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 5584, pp. 73–79. Springer, 2009.
- [24] A. Semenov, O. Zaikin, I. Otpuschennikov, S. Kochemazov, and A. Ignatiev. On cryptographic attacks using backdoors for SAT. In *Proc. 32nd AAAI Conference on Artificial Intelligence (AAAI)*, pp. 6641–6648, 2018.
- [25] G. S. Tseytin. On the complexity of derivation in propositional calculus. In *Structures in Constructive Mathematics and Mathematical Logic, Part II*, pp. 115–125. Consultants Bureau, New York, 1968.

Déclaration d'utilisation de l'IA